

KAVANA RouteFleet

Plataforma de gestión de repartos para empresas de reparto. Permite a los repartidores escanear albaranes, entregar con firma del cliente (POD) y a las oficinas hacer seguimiento de repartos por repartidor, ver firmas y KPIs.

Arquitectura (todo fuera del VPS)

Componente	Donde vive	URL
Backend API (Node/Express, store JSON)	Render	https://routefleet-api.onrender.com
App del repartidor (React PWA)	GitHub Pages (/ app)	https://routefleet.kavanasystems.com/app
Panel oficinas "Torre de Control" (React)	GitHub Pages (/ , rama gh-pages-admin)	https://routefleet.kavanasystems.com

El VPS solo se usa para desarrollo/documentación. Los proyectos terminados viven en servicios externos (Render + GitHub Pages), igual que CleanStock.

Estructura del repo

```
server/           Backend Express + store JSON (sin SQLite)
  src/db.js       Capa de datos (stops, incidents, drivers, pods)
  src/routes/api.js Endpoints REST
  src/services/   pdfService (POD), ocrService, aiService
  tests/         36 tests (node --test) incl. autenticación JWT
client/           App del repartidor (Vite + React, base /app/)
client-admin/     Panel de oficinas Torre de Control (Vite + React, base
.github/workflows/ deploy-combined.yml (panel + app en gh-pages-admin)
```

Desarrollo rápido

```
# Backend
cd server && npm install && npm test          # 36 tests verdes (incl. JWT)
ROUTEFLEET_DB=/tmp/dev.json PORT=5001 node src/index.js

# App repartidor
cd client && npm install && npm run dev        # http://localhost:5173

# Panel oficinas
cd client-admin && npm install && npm run dev # http://localhost:5173
```

Deploy

- **Backend:** Render (Web Service, rama `main`, Root `server`, start `node src/index.js`). Variables obligatorias: `JWT_SECRET` (cadena fuerte y aleatoria ≥ 32 chars — firma los JWT). `OFFICE_PIN` (PIN oficina, def. `0000`), `CORS_ORIGINS` (def. `github.io`).
- `routefleet.kavanasystems.com`). Todos los endpoints exigen JWT (oficina/driver).
- **Frontends:** un único workflow `deploy-combined.yml` builda app y panel y publica en `gh-pages-admin` (panel en `/`, app en `/app`). Secret repo `VITE_API_BASE=https://routefleet-api.onrender.com`. En Settings → Pages: rama `gh-pages-admin`, custom domain `routefleet.kavanasystems.com`.
- **DNS:** CNAME `routefleet` → `kavanasystemsinfo-ui.github.io`.

Documentación

Ver `docs/` (ARQUITECTURA.md, BACKEND.md, APP_REPARTIDOR.md, PANEL_OFICINAS.md, DESPLIEGUE.md, API.md).

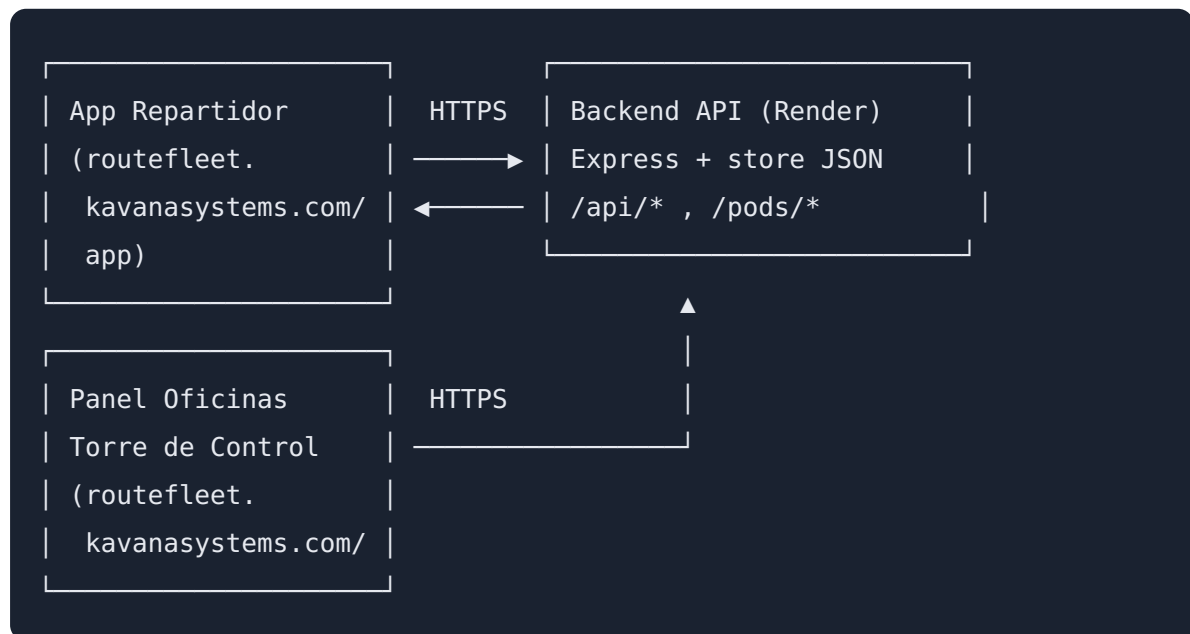
Arquitectura — KAVANA

RouteFleet

Principio rector

El VPS (167.233.97.71) es **solo entorno de desarrollo y documentación**. Los proyectos terminados viven fuera del VPS, igual que CleanStock: - Backend → **Render** (servicio `routefleet-api`, Free). - Frontends → **GitHub Pages** (sin servidor propio, HTTPS automático).

Componentes



Datos

- Store JSON en disco (`routefleet.json` en el server de Render). Sin SQLite para evitar compilación nativa en Render Free. Listo para migrar a Postgres.
- PODs: PDFs generados con pdfkit en `./pods`. En Render son efímeros tras spin-down; el endpoint `GET /api/stops/:id/pod` los regenera al vuelo si la parada tiene firma guardada.

Identidades y autenticación (JWT)

- **Repartidor:** se identifica con un PIN (4 dígitos) que se guarda en `localStorage` del móvil. `POST /api/drivers/login` valida el PIN y devuelve un **JWT** (`role:driver` , exp 8h). Cada llamada del repartidor (crear parada, entregar, incidencia, borrar) envía ese JWT en ``Authorization: Bearer ***``.
- **Oficina:** login con PIN único de empresa (`OFFICE_PIN` , def. `0000`). `POST /api/office/login` devuelve un **JWT** (`role:office`). El panel lo guarda en `sessionStorage` y lo envía en cada fetch.
- **Firma:** HS256 con `crypto` nativo de Node (sin dependencias externas). Secreto en `JWT_SECRET` (env de Render). Sin token → `401` ; rol incorrecto → `403` .
- MVP sin usuarios/roles múltiples (YAGNI): solo dos roles (office / driver).

Seguridad

- Toda lectura del panel (repartos, firmas, KPIs) y toda escritura del repartidor exigen JWT válido. Nadie puede leer los datos de los clientes sin autenticarse.
- El POD se sirve también con `?token=` para los enlaces/iFrame del panel (GET).
- `JWT_SECRET` debe configurarse en Render con un valor fuerte y aleatorio; si no se define, se usa un fallback de desarrollo (válido pero no para producción).

Flujo

1. Oficina da de alta repartidores (PIN) en el panel.
 2. Repartidor abre la app, introduce su PIN (se guarda en el móvil).
 3. Repartidor escanea albarán → parada creada con `driver_id` .
 4. Repartidor entrega → POD generado en el navegador (jsPDF) + subido al backend.
 5. Oficina ve en el panel: Dashboard (KPIs), Repartos (filtros), Firmas (galería).
-

Backend — KAVANA RouteFleet

Node/Express, store en JSON (sin SQLite). Todo desplegado en Render (`routefleet-api`). **36 tests** con `node --test` (incluye autenticación JWT).

Arranque

```
cd server
npm install
npm test           # 36 tests verdes (incl. JWT)
ROUTEFLEET_DB=/tmp/dev.json PORT=5001 node src/index.js
```

Variables de entorno

Var	Def.	Uso
<code>PORT</code>	5001	Puerto
<code>ROUTEFLEET_DB</code>	<code>./routefleet.json</code>	Ruta del store JSON
<code>JWT_SECRET</code>	<code>routefleet-dev-secret-change-me</code>	Obligatoria en prod: clave HS256 para firmar JWT (usar cadena ≥ 32 chars aleatoria)
<code>OFFICE_PIN</code>	<code>0000</code>	PIN de login de oficina
<code>CORS_ORIGINS</code>	github.io + routefleet.kavanasystems.com	Orígenes CORS

Autenticación (JWT HS256)

- `src/auth.js` : firma/verifica con `crypto` nativo (sin dependencias). `signToken(payload)` , `verifyToken(token)` , `extractToken(req)` (header `Authorization: Bearer ***` o query `?token=`), `requireAuth(roles)` `.
- `POST /api/office/login` → `{token}` (role `office` , exp 8h).
- `POST /api/drivers/login` → `{token, driver}` (role `driver`).
- Todos los endpoints de datos exigen JWT (ver `API.md`). Sin token → 401 ; rol incorrecto → 403 .

Capa de datos (`src/db.js`)

Store JSON con interfaz agnóstica (`initDb()` , `queries.*`): - `stops` : [{id, stop_number, address, status, driver_id, receiver_name, signature, created_at}] - `drivers` : [{id, name, pin, phone, active}] - `incidents` : [{id, stop_id, type, photo_data, notes, created_at}] - `settings` : {cost_per_km, cost_per_hour} - `Pods` : {[stopId]: url}

Servicios

- `pdfService.js` : genera el POD (PDF) con firma negra sobre blanco.
- `ocrService.js` : procesa imagen de albarán (stub/IA).
- `aiService.js` : optimización de ruta (stub/IA).

API REST — KAVANA RouteFleet

Base: `https://routefleet-api.onrender.com/api`

Auth: todos los endpoints exigen `Authorization: Bearer ***` (JWT).
Roles: `office` (panel) y `driver` (app repartidor). Sin token → 401 ; rol incorrecto → 403 . El PIN de oficina es `OFFICE_PIN` (def. `0000``).

Paradas (stops)

- `GET /stops` → lista (role `office` o `driver`). Query: ?
`driver_id=1&status=delivered&from=2026-07-01&to=2026-07-31`
- `POST /ocr_manual` → crea parada (role `driver`). Body: {stop_number, address, driver_id}
- `PATCH /stops/:id` → actualiza (role `driver`). Body: {status, signature, receiverName, driver_id} . Si `status=delivered` y `signature` , genera POD y devuelve {success, pod_url} .
- `DELETE /stops/:id` (role `driver`) · `DELETE /stops` (role `driver` , borra todas)
- `POST /stops/:id/incident` (role `driver`) → Body {type, photo_data, notes} (pone `status=incident`)

- `GET /stops/:id/pod` → `{pod_url}` (role `office` o `driver` ; también ? token=)

Repartidores (drivers)

- `GET /drivers` (role `office`) · `POST /drivers` (role `office`) → Body `{name, pin, phone}`
- `PATCH /drivers/:id` (role `office`) → Body `{active: bool}`
- `POST /drivers/login` → Body `{pin}` . 200 + `{token, driver}` si el PIN es válido; 401 si no.

Oficina

- `POST /office/login` → Body `{pin}` . 200 + `{token}` si coincide con `OFFICE_PIN` ; 401 si no.

Config / métricas

- `GET /settings` (role `office`) · `PUT /settings` (`{cost_per_km, cost_per_hour}`)
- `GET /dashboard-data` (role `office`) → `{metrics, stops (con pod_url), settings}`

PODs (estáticos)

- `GET /pods/<file>.pdf` → descarga del PDF
-

App del Repartidor — KAVANA RouteFleet

React PWA (carpeta `client/`). Desplegada en GitHub Pages en `routefleet.kavanasystems.com/app/` (subpath `/app`). Estética oscura industrial, pensada para móvil.

Pantalla de identificación (PIN + JWT)

Al abrir la app, si no hay token guardado, pide el PIN del repartidor. `POST /api/drivers/login` valida el PIN y devuelve un **JWT** que se guarda en `localStorage` (`routefleet_driver_token`). Todas las llamadas del repartidor envían ese JWT en ``Authorization: Bearer ***``.

Funciones

- **Carga:** escáner de albarán (cámara) → `POST /api/ocr` (IA) → crea parada con `driver_id` vía `POST /api/ocr_manual` (requiere JWT driver).
- **Mapa:** vista de la parada activa (Google Maps embed).
- **Entregar:** firma del cliente en canvas (blanco, trazo negro) → `PATCH /stops/:id` con firma (requiere JWT driver) → genera POD en el navegador (jsPDF) y descarga garantizada (`DESCARGAR POD`). También sube la firma al backend.
- **Incidencia:** foto + nota → `POST /stops/:id/incident` (requiere JWT driver).

Variables

- `VITE_API_BASE` (build-time) → `https://routefleet-api.onrender.com` Sin slash final; el cliente añade `/api`.

Build

```
cd client && npm install && npm run build # → dist/ (base /app/)
```

Panel de Oficinas — Torre de Control

React (carpeta `client-admin/`). Desplegado en GitHub Pages en la rama `gh-pages-admin` , con dominio propio `routefleet.kavanasystems.com` (raíz

/). Estética backoffice, sidebar + tablas densas (pensado para escritorio).

Tema (Kavana / Clásico)

El panel ofrece dos temas conmutables por pestañas en el sidebar: - **Kavana**: oscuro industrial (estética de marca). - **Clásico**: claro estilo CRM (azul corporativo sobre blanco), **predeterminado** para oficinas. La elección se persiste en `localStorage` (`rf_admin_theme`).

Login (JWT)

Pantalla de PIN de oficina (`POST /api/office/login`) → devuelve un **JWT** que se guarda en `sessionStorage` y se envía en cada fetch. Botón "Salir" limpia la sesión. PIN por defecto `0000` (cambiable en Render con `OFFICE_PIN`).

Secciones

- **Dashboard**: KPIs (total / entregados / pendientes / incidencias / OPEX estimado) + barras de entregas por repartidor (%).
- **Repartidores**: alta (nombre, PIN, teléfono) y activar/desactivar.
- **Repertos**: tabla con filtros por repartidor, estado y rango de fechas. Enlace a POD por parada (con `?token=`).
- **Firmas**: galería de firmas de clientes (iframe del POD + descarga PDF), filtrable por repartidor y fechas.
- **Incidencias**: lista de incidencias reportadas por los repartidores.

Variables

- `VITE_API_BASE` (build-time) → `https://routefleet-api.onrender.com`

Build

```
cd client-admin && npm install && npm run build # → dist/ (base /)
```

Despliegue — KAVANA

RouteFleet

Arquitectura de despliegue (todo fuera del VPS)

Componente	Donde vive	URL
Backend API (Node/Express, store JSON)	Render	https://routefleet-api.onrender.com
Panel oficinas "Torre de Control" (React)	GitHub Pages (gh-pages-admin)	https://routefleet.kavanasystems.com
App del repartidor (React PWA)	GitHub Pages (gh-pages-admin / /app)	https://routefleet.kavanasystems.com/app

El VPS solo se usa para desarrollo/documentación.

1. Backend (Render)

- New → Web Service → repo `kavana-RouteFleet` , rama `main` , Root `server` .
- Build: `npm install` · Start: `node src/index.js` .
- Variables obligatorias: `JWT_SECRET` (cadena ≥ 32 chars aleatoria), `OFFICE_PIN` (PIN oficina, def. `0000`), `CORS_ORIGINS` (def. `github.io + routefleet.kavanasystems.com`).
- **Nota:** el auto-deploy puede quedar fijado a un commit viejo. Tras push de cambios al backend, pulsa **Manual Deploy** para aplicar `main` HEAD.

2. Frontends combinados (GitHub Pages + dominio propio)

- **Un solo workflow** `.github/workflows/deploy-combined.yml` builda los dos frontends y publica en la rama `gh-pages-admin` :
- Panel de oficinas en la **raíz** (/).
- App del repartidor en `/app` (`client/` con `base: '/app/'`).
- Se incluye `.nojekyll` .
- En repo → **Settings** → **Pages**:
- Source: **Deploy from a branch**
- Branch: `gh-pages-admin` , carpeta `/` (root)
- Custom domain: `routefleet.kavanasystems.com` → Save → **Enforce HTTPS**
- **Si el dominio sirve contenido viejo tras un deploy**: quita (Remove) el custom domain, guarda, espera 1 min y vuelve a ponerlo + Enforce HTTPS. Esto reasocia el dominio al sitio de `gh-pages-admin` .

3. DNS (gestor del dominio `kavanasystems.com`)

- CNAME `routefleet` → `kavanasystemsinfo-ui.github.io` .

Verificar tras desplegar

```
# Backend vivo
curl https://routefleet-api.onrender.com/api/settings
# Auth: sin token debe dar 401
curl -o /dev/null -w "%{http_code}\n" https://routefleet-api.onrender.com/
# CORS para panel
curl -s -i https://routefleet-api.onrender.com/api/drivers \
  -H "Origin: https://routefleet.kavanasystems.com" \
  | grep access-control-allow-origin
```